

Module 6

Topics: Re-naming, labeling, and formatting variables, Useful procedures, Output delivery system basics

Files to download: mod6.sas7bdat

6.1 - Renaming and labeling variables in SAS

To rename a variable, use the RENAME statement. RENAME OLD NAME = NEW NAME. For example, in our data set we want to rename the variable "ILL" to "CACO".

```
LIBNAME NICOLE 'H:\AEPI537D';

DATA TEMP;
    SET NICOLE.mod6;
    RENAME ILL = CACO;
RUN;

PROC FREQ DATA = TEMP;
    TABLES CACO;
RUN;
```

Here is another way you can rename a variable. In this example will create the exact same TEMP dataset from mod6 but use a slightly different renaming method. Several things we have learned (RENAME, DROP, KEEP) can be done this way. The first method is better for simple data steps like we have here, the second method is better when merging datasets.

```
DATA TEMP;
    SET NICOLE.mod6 (RENAME=(ILL = CACO));
RUN;

PROC FREQ DATA = TEMP;
    TABLES CACO;
RUN;
```

It is helpful when reading output to give your variables labels that describe more completely the data contained in the variable. For example, the variable SEX1 might be labeled GENDER and the variable WT3LEV might be labeled WEIGHT CATEGORY.

```
DATA TEMP2;
    SET TEMP;
    LABEL SEX1 = 'GENDER'
           WT3LEV = 'WEIGHT CATEGORY';
RUN;
```

Let's take a look at the output now that we have assigned some labels to these variables.

```
PROC FREQ DATA = TEMP2;
```

```
TABLES SEX1*WT3LEV;
RUN;
```

Before going any further, let's save the changes that we have made to a permanent dataset.

```
DATA NICOLE.mod6_rev;
    SET TEMP2;
RUN;
```

6.2 - Formatting variables in SAS

Another way to provide more information in SAS output is to assign formats to the variables in your dataset. A format is used to assign names to each category of your categorical variables that will be printed in the output in place of the stored value. SAS has many formats that it keeps in a special library called LIBRARY. You have already worked with one of these formats in this class, **MMDDYY10..** You will recall that you used this format to display dates in a way that you are used to seeing them rather than the way that SAS stores dates internally. All SAS formats must have a period at the end. This tells SAS that you are referring to a format.

In addition to these formats that are already available in the SAS format library, you can create your own for use with other variables. For example, for the variable CACO if you defined your cases as 1 and your controls as 0, you can use a format to have SAS print the words CASE and CONTROL rather than 1 and 0 in your output. This makes the output easier to read and interpret. However, formats are different from character variables. Formats do not change the numeric nature of the variable, but rather assign names to the numeric categories that are used in the output of procedures.

Once you have created one or more formats, there are two ways that they can be used: 1) as part of a procedure in which case they are referred to as temporary formats; or 2) a format can be permanently assigned to specific variables in a new SAS dataset.

Let's first explore a couple of ways that we can use temporary formats. There are several variables in our dataset that are coded 1 for "Yes" and 0 for "No" including those for smoking, plane travel and vitamin use. We can create a format that can be used for procedures involving these variables.

```
PROC FORMAT;
    VALUE YESNOF
        0='NO'
        1='YES';
RUN;
```

When you execute this program, SAS will tell you that "Format YESNOF has been output." This means that SAS has stored this format in a temporary library and it can be used at any time during the current SAS session. Now let's use the format in a procedure.

```
PROC FREQ DATA=TEMP2;
    TABLES (SMOKE PLANE VITAMIN)*WT3LEV;
```

```
FORMAT SMOKE PLANE VITAMIN YESNOF.;
RUN;
```

Notice that when you use the name of a format in a format statement, the format name must be followed by a period. SAS will not recognize a format name without this punctuation.

Also notice that we have some data that we haven't cleaned up yet in the VITAMIN variable. There are values in the dataset that are greater than 1. You will be asked to correct these data later in the lab.

Another way that you might use a temporary format is to create categories for a continuous variable. For example, we might want to look at the distribution of smoking prevalence across various age groups. We can create age groups using a format as shown in the following example.

```
PROC FORMAT;
  VALUE AGEFMT
    15 - 24 = '<25'
    25 - 29 = '25-29'
    30 - 100 = '30+';
RUN;

PROC FREQ DATA=TEMP2;
  TABLES ENTRYAGE*SMOKE;
  FORMAT ENTRYAGE AGEFMT.;
RUN;
```

Tip: Note that when you don't know the upper or lower limit for making a format you can use the words HIGH and LOW instead. For example in the format above I could have used LOW - 24 = '<25' or 30 - HIGH = '30+';.

Notice that you specify the continuous variable ENTRYAGE in the TABLES statement but the format statement causes the output to take on the categories that you specified when you created the AGEFMT format.

When formats are created permanently in a dataset they are written to a library called LIBRARY. The LIBRARY library must be accessed each time the data set is accessed. The following program illustrates how permanent formats are created.

```
LIBNAME LIBRARY 'H:\AEPI537D';    *creates a format library on the H: drive;

PROC FORMAT LIBRARY = LIBRARY;    *assigns the place to save the formats below;

  VALUE GENDERF
    1='FEMALE'
    0='MALE';

  VALUE SMOKERF
    0='NO'
    1='YES';

RUN;
```

Now that you have created the format file that contains these formats, let's create a new permanent dataset and PERMANENTLY assign these formats to a couple of variables in our dataset.

```
DATA NICOLE.mod6_fmt;
    SET TEMP2;
    FORMAT SEX1 GENDERF. SMOKE SMOKERF.;
RUN;
```

To see the difference between the formatted tables and the non-formatted tables, we will run a PROC FREQ on these variables first in the mod6_rev dataset, then the same FREQ on the new mod6_fmt dataset that contains the formatted variables. To do this, you must first access the format library. You can look in your "library" library using the SAS explorer and see there is now a data set called formats. *Note that I am using the same pathway ('H:\AEPI537D') for my dataset and format library – you could have these in different locations as long as you tell SAS where to find both the dataset and the format library.*

```
LIBNAME NICOLE 'H:\AEPI537D';
LIBNAME LIBRARY 'H:\AEPI537D';

PROC FREQ DATA=NICOLE.mod6_rev;
    TABLES SEX1*SMOKE;
RUN;

PROC FREQ DATA=NICOLE.mod6_fmt;
    TABLES SEX1*SMOKE;
RUN;
```

TWO NOTES OF CAUTION:

1. Do not make your formats (i.e. the words) too long or it will not nicely fit into the table cell but rather will wrap around.
2. Keep in mind that if you create a permanent SAS dataset with formats assigned to the variables, you MUST have the formats catalog available to SAS whenever you want to use the dataset. If the formats catalog (the file called formats.sas7bcat) is not available, SAS will not let you use the dataset at all.
 - If you ever receive a formatted dataset from someone else with no formats catalog, you can trick SAS into letting you open the dataset by using the following statement at the beginning of your program: `options nofmterr;`

6.3 - Introduction to the SAS Output Delivery System (ODS).

The general output of SAS data and procedures, called listing output, uses fonts and symbols that were designed for line printers. With today's modern printers, this output may not meet users' needs for higher quality printing and display. The SAS output delivery system (ODS) was developed to overcome these limitations.

Like most features in SAS, the various uses and abilities of this system are quite extensive and allow for a large degree of control over the output from SAS procedures and programs. This section provides a very brief introduction to the ODS system and students are encouraged to explore the capabilities of this system in the SAS documentation.

The ODS provides the ability to send the output of SAS procedures to files in several standard formats including RTF files for use with Microsoft Word and PDF files for use with Adobe Reader. The following example illustrates the use of the ODS system to produce a document in the RTF format.

```
ODS RTF FILE='H:\AEPI537D\SAMPLE.RTF';
PROC FREQ DATA=TEMP2;
    TABLES ENTRYAGE*SMOKE;
    FORMAT ENTRYAGE AGEFMT.;
RUN;
ODS RTF CLOSE; *You must submit the close before you will be able to view the output file;
```

You can also suppress the results in the output window and send your results only to a file by using the NORESULTS command as shown in the next program that now designates that the output file will be in PDF format.

Let's take a look at the standard ODS output associated with a FREQ procedure using the PDF output capabilities of the output delivery system. We can run the frequency procedure that we previously used on the mod6_fmt dataset that has permanent formats assigned to the variables SEX1 and SMOKE directing the output to a PDF file using ODS as follows.

```
ODS NORESULTS;
ODS PDF FILE = 'H:\AEPI537D\SAMPLE.PDF';
PROC FREQ DATA=NICOLE.mod6_fmt;
    TABLES SEX1*SMOKE;
RUN;
ODS PDF CLOSE;
```

As mentioned earlier, this is only an introduction to the ODS and there is much more functionality to this system than is demonstrated here. In version 9.2 of SAS, ODS has been integrated into the SAS/GRAPH procedures to allow a greater flexibility in modifying and customizing the output of these programs.

***Note:** If you use ODS NORESULTS this will apply to future procedures. **Use ODS RESULTS to turn this feature back on.**

You can customize your own style templates or choose from the ones SAS offers:
Proc template; **list** styles; **RUN;**

Example: **ODS PDF FILE = 'H:\AEPI537D\SAMPLE.PDF' style=journal;**

6.4 - Database management tools: the COMPARE and TRANSPOSE procedures.

PROC COMPARE compares values of variables in two SAS data sets and reports the differences found. PROC COMPARE is especially useful in database management (e.g. to compare updated versions of a dataset or to assess the results of double key data entry of raw data).

To illustrate how COMPARE works, let's make a change to the data in the mod6_rev dataset by dropping a couple of variables and changing the case/control status of one of the subjects. Then we will compare the datasets using PROC CONTENTS and PROC COMPARE to see the differences in how these procedures work.

```
LIBNAME NICOLE 'H:\AEPI537D';

DATA TEMP3(DROP=KIDS HOUSE);
  SET NICOLE.mod6_rev;
  IF STUDYID=29 THEN CACO=0;
RUN;

PROC CONTENTS DATA = NICOLE.mod6_rev;
RUN;

PROC CONTENTS DATA = TEMP3;
RUN;
```

You can see that the TEMP3 dataset has 2 fewer variables by comparing the two datasets using the CONTENTS procedure but you have to do the comparison manually to determine which variables are missing from TEMP3. Of course, since the CONTENTS procedure does not look at individual values of the variables, there is no way that we can identify that the data have been changed for the subject with STUDYID of 29 using this procedure. Now let's see how PROC COMPARE performs the same task.

```
PROC COMPARE DATA = NICOLE.mod6_rev COMPARE = TEMP3;
  ID STUDYID;
RUN;
```

Not only does the COMPARE procedure tell you that the number of observations is the same in each dataset and that there are changes to the number of variables, but COMPARE also looks at the data stored in every variable for each observation in each dataset and compares it to determine whether or not the data have changed from one dataset to another. In this case, the procedure correctly identifies the changed data in case control status and because you specified the ID variable (ID STUDYID) the compare procedure will give you the STUDYID of the observation in which the data were changed. In addition, when you specify the LISTVAR option, the procedure will tell you which variables are different in the two datasets. Other options are available with this procedure and, as always, you are encouraged to learn more about this procedure in SAS help.

Another very useful database management procedure is PROC TRANSPOSE. This procedure reads a SAS data set and creates a new dataset as its only output. The rows of the original data matrix become columns, and the columns become rows. Variables in the new dataset correspond to observations in

the old data set. You may want to transpose a dataset to get it in a form for analysis. For example, the following data set contains information on five subjects who have had systolic blood pressure measurements taken on four occasions.

```
DATA DBP;
INPUT STUDYID BP;
DATALINES;
01 126
01 131
01 135
01 124
02 145
02 144
02 140
02 142
03 150
03 148
03 152
03 160
04 120
04 119
04 115
04 120
05 125
05 130
05 135
05 128
;
```

```
PROC PRINT DATA = DBP; RUN;
```

If we transpose the data set directly, the 20x2 matrix becomes a 2x20 matrix.

```
PROC TRANSPOSE DATA = DBP OUT = DBP2;
RUN;
```

```
PROC PRINT DATA = DBP2; RUN;
```

A more useful way to do this is to transpose the original data set (DBP) by STUDYID and rename the columns.

```
PROC TRANSPOSE DATA = DBP OUT = DBP3
  (RENAME = (COL1 = TIME1 COL2 = TIME2 COL3 = TIME3 COL4 = TIME4));
  BY STUDYID;
  VAR BP;
RUN;

PROC PRINT DATA = DBP3;
RUN;
```

The resulting dataset is in a more useful form for the typical types of analyses you may wish to perform.

Bonus Output File Option

Sometimes it can be useful to stamp output files with the date they were created. There are several different ways to do this, but here is one example.

At the beginning of your program you can create macro date fields that can be used later on.

```
data _NULL_;  
    call symput ("db", put(today(), date9.));  
run;
```

The data null step isn't creating a dataset, just a variable we can use. Now if I want to append the date to a file I am creating I can do something like this:

```
ods pdf file = "H:\AEPI537D\ANOTHERSAMPLE_&db..pdf" style = journal ;  
    PROC PRINT DATA = DBP3; RUN;  
ods pdf close;
```

Note that the file name is appending the value of DB that we creating in the data null step by using an &.

MOD 6: USE SAS COMMENTS TO NUMBER YOUR WORK.

1. Make a temporary SAS dataset called `TEMPLAB` from `mod6_rev` and do the following:
 - a. Create a descriptive label for the variable `ENTRYAGE` indicating that it is the participant's age at enrollment, and a label for the variable `KIDS` indicating that it is the total number of kids in the household.
 - b. Create a temporary format for the continuous variable `ENTRYAGE` with 5 categories (categories are up to you).
 - c. Create a temporary format for the variable `MARITAL1` where the value 0 is "Married" and the value 1 is "Unmarried."
2. Run a procedure step to look at the formatted version of `ENTRYAGE`, and the formatted version of `MARITAL1`. Use the output data system (ODS) to direct the output of the procedure to an RTF file on the H: drive.
3. Transpose the following dataset of test scores so that you end up with one row per ID that has each test score as separate columns called `SCORE1` `SCORE2` and `SCORE3`.

```

DATA TPOSE;
INPUT ID SCORE;
DATALINES;
1 90
1 87
1 92
2 80
2 90
2 50
3 75
3 95
3 95
4 90
4 91
4 82
5 80
5 97
5 90
6 75
6 95
6 95
7 90
7 91
7 82
8 80
8 97
8 90
;
RUN;

```